

FAILED PAYMENTS · ABANDONED REVENUE

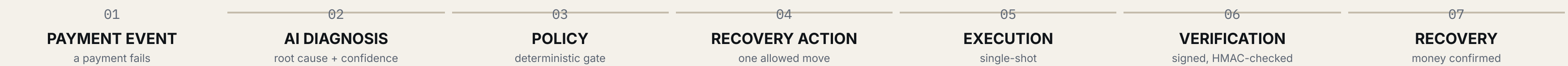
Failed payments aren't always lost revenue.



THE PRODUCT

RecoveryOS closes the loop on lost revenue.

From a failed payment event to a **verified** recovery — one continuous, measurable lifecycle. Not a retry. A recovery pipeline.



AI proposes. Policy decides. Recovery is verified.

Intelligence proposes. Deterministic policy authorizes. Evidence confirms the money came back.

One operator console. A batch, not a hack.

RecoveryOS works across 500 recovered-and-evaluated payment events, every state derived from a single persisted decision chain and a signed Razorpay webhook.

Recovery Command Center • live persisted state

SIMULATED

REAL_RAZORPAY • Test Mode

Revenue at Risk

₹53,40,990

sum of ingested failed payments

Simulated Revenue Recovered

₹9,20,207

SIMULATED evaluation • V2 strategy

Recovery Rate

17.0%

SIMULATED • across 500 events

Blocked Interventions

1,685

denied by deterministic policy

EVENT / CATEGORY	AMOUNT	FAILURE	DIAGNOSIS	POLICY	STATE
evt_p14traceC_...50Z_02	₹4,999	expired card • renewal failed	customer_action_needed • 87%	ALLOW	RECOVERED webhook verified
evt_p14live_...55Z_01	₹4,999	expired card	customer_action_needed • 84%	ALLOW	RECOVERED
fraud-suspect events • batch	—	suspicious pattern	fraud_suspect	DENY	BLOCKED • 1,285
recent-intervention events • batch	—	transient failure	transient	DENY	BLOCKED • cooldown 30m

THE CORE INSIGHT

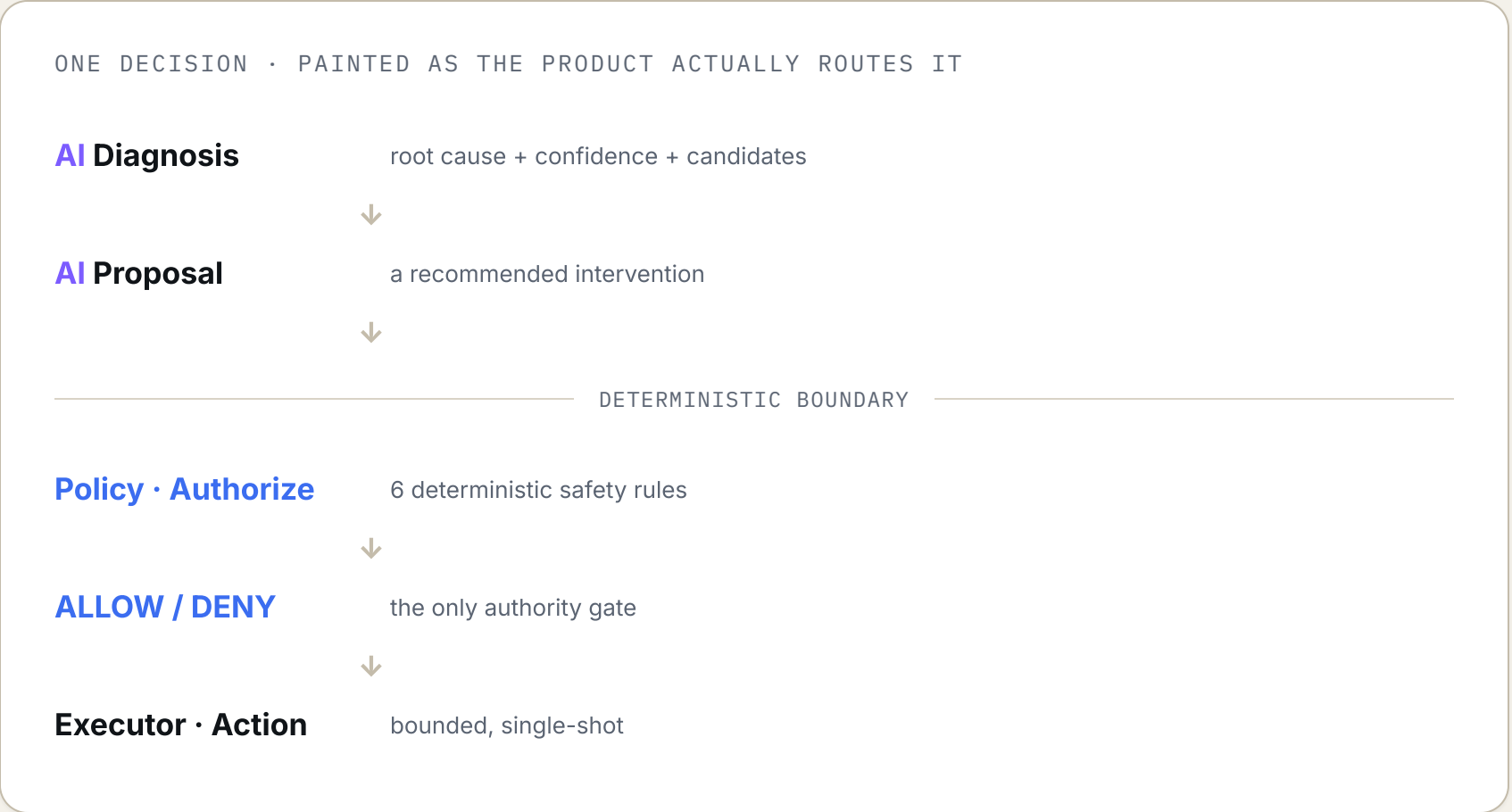
AI handles ambiguity. Policy retains authority.

The model reasons about *why* a payment failed and *what* might work. It never touches money, never finalises an action.

- ADVISORY

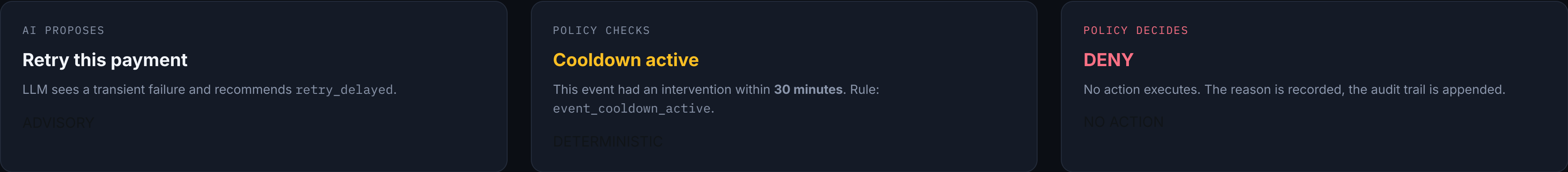
LLM output is advisory
- AUTHORITATIVE

Authorization is deterministic



A good recovery system knows when NOT to act.

The model cannot override the gate. Example from live policy state — a single real denial reason, already enforced across 1,685 blocked decisions.



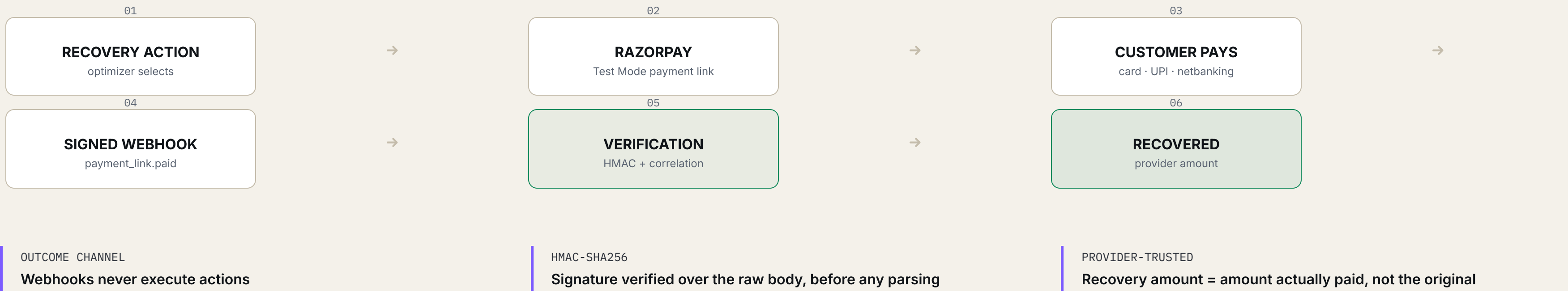
AI recommends → policy evaluates → DENY → nothing executes

A safe denial is not a failure of RecoveryOS – it is the system working correctly. The model is bounded. Authority stays deterministic.

Fraud interventions: 0 · Unauthorized executions: 0 · across every benchmark seed

Sending the action isn't the same as recovering the money.

The loop is only closed when Razorpay confirms the payment. RecoveryOS verifies the resulting state — it never assumes success.



OUTCOME CHANNEL
Webhooks never execute actions

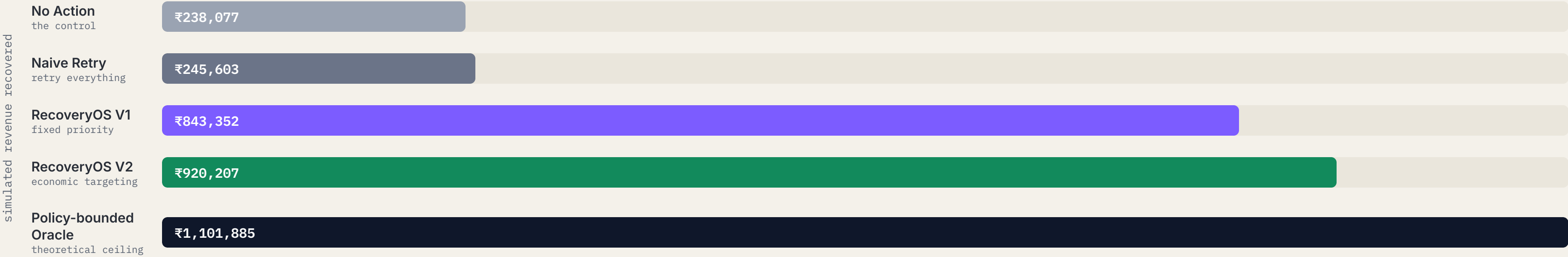
HMAC-SHA256
Signature verified over the raw body, before any parsing

PROVIDER-TRUSTED
Recovery amount = amount actually paid, not the original

Same 500 failed payments. Three strategies.

RecoveryOS V2 against No Action and a naive blanket retry — identical events, identical hidden outcome model. Simulated revenue recovered.

● SIMULATED EVALUATION — not production revenue



+₹6,82,130

RecoveryOS V2 vs No Action
simulated incremental revenue

93.1%

Oracle value capture
V2 · vs 79.4% (V1) · vs 4.3% (Naive)

0

Fraud / unauthorized actions
targeting win, not restlessness — V2 uses 180 interventions vs Naive's 240

Serious engineering, where it matters for money.

Six guarantees modeled on real-world financial constraints — not a demo feature list.

■

Deterministic authority

Six rules — fraud, terminal, duplicate, customer limit, cooldown, spend cap. Fixed order, fail-closed, and the optimizer can never widen the allowed set.

```
policy_gate :: LLM_output → ALLOW/DENY
```

■

Append-only audit trail

Every recommendation, decision, and execution is recorded — decisions are written **before** they execute, and the full loop is reconstructable.

```
decisions · claims · outcomes → immutable
```

■

Idempotency

A durable claim keyed to (event, intervention) decides which single attempt crosses the money-moving boundary. No double-charges, no double-spend.

```
PK(event_id, intervention) → one side-effect
```

■

Bounded spend

Intervention costs and a daily spend cap bound what RecoveryOS can spend chasing a recovery — plus a 30-minute cooldown and 2-per-customer limit.

```
daily_spend_cap = ₹50,000 · cooldown 30m
```

■

Fail-closed verification

Signatures verified over the raw body before any parsing. An invalid or missing webhook signature can never change state.

```
HMAC-SHA256 · constant-time · reject 4xx
```

■

Stateful recovery lifecycle

Recovery is a tracked state machine — not a fire-and-forget call. Outcomes move waiting → pending → verified only on evidence.

```
PENDING_OUTCOME → RECOVERED
```

We know exactly what we built. We know what remains.

A real control plane running today — validated against real Razorpay infrastructure in Test Mode. Batch economics are honestly labelled simulated.

REAL · implemented & running

WORKING TODAY

- ✓ Control-plane runtime — diagnosis → policy → action → verification
- ✓ Razorpay Test Mode integration — REAL_RAZORPAY Payment Link
- ✓ Signed webhook verification — a genuine payment recovered
- ✓ Operator console & decision trace
- ✓ Audit trail, policy engine, spend limits

SIMULATED · evaluated

BENCHMARK

- Batch economics — the ₹9,20,207 chart
- Large-scale recovery evaluation — 500 synthetic events
- Signal-bearing hidden world (phase 17 methodology)

No production revenue claims. No unattended money movement.
That boundary is the point — it keeps the product honest and the engineering credible.

RecoveryOS

AI proposes. Policy decides. **Recovery is verified.**

BOUNDED · LOGGED · PROVABLE

From Test Mode validation → real merchant recovery.